

Privora

Setup & Deployment Guide

Prepared for: Bassam — Founder, Privora
April 2026 | Confidential

Privora — Setup & Deployment Guide

Overview

This guide covers setting up Privora from scratch — locally for development and on a server for production.

Tech Stack - Backend: Node.js 20 + Express.js - Database: PostgreSQL 15+ (via Sequelize ORM) - Frontend: React 18 + Vite + TypeScript + Tailwind CSS - Auth: JWT + OTP (nodemailer for email delivery)

Prerequisites

Tool	Minimum Version	Install
Node.js	20.x LTS	https://nodejs.org
npm	10.x	Included with Node
PostgreSQL	15+	https://postgresql.org
Git	any	https://git-scm.com

1. Clone the Repository

```
git clone https://github.com/0xBassam/Bassam.git
cd Bassam
```

2. Backend Setup

2.1 Install Dependencies

```
cd backend
npm install
```

2.2 Create Environment File

```
cp .env.example .env
```

Edit `.env` with the following values:

```
# Application
NODE_ENV=development
PORT=5000

# Database (PostgreSQL)
DB_URL=postgresql://privora_user:yourpassword@localhost:5432/
privora_dev
DB_DIALECT=postgres

# JWT Secrets (generate with: openssl rand -hex 64)
JWT_SECRET=your_jwt_secret_min_64_chars
JWT_REFRESH_SECRET=your_refresh_secret_min_64_chars
JWT_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d

# OTP
OTP_SECRET=your_otp_secret
OTP_EXPIRES_MINUTES=10

# Email (SMTP – use SendGrid or AWS SES in production)
SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
SMTP_USER=apikey
SMTP_PASS=your_sendgrid_api_key
EMAIL_FROM=no-reply@privora.sa

# Frontend URL (for CORS)
CLIENT_URL=http://localhost:5173

# Rate Limiting
RATE_LIMIT_OTP_MAX=5
```

2.3 Create PostgreSQL Database

```
# Connect to PostgreSQL
psql -U postgres

# Create user and database
CREATE USER privora_user WITH PASSWORD 'yourpassword';
CREATE DATABASE privora_dev OWNER privora_user;
GRANT ALL PRIVILEGES ON DATABASE privora_dev TO privora_user;
\q
```

2.4 Run Database Migration

Sequelize will auto-sync tables on first start in development mode. To run manually:

```
npm run db:migrate
```

2.5 Start Backend

```
# Development (with auto-reload)
npm run dev

# Production
npm start
```

Backend will be available at: `http://localhost:5000`

API health check: `GET http://localhost:5000/api/v1/health`

2.6 Run Tests

```
npm test
```

All 156 integration tests should pass. Tests use an in-memory SQLite instance — no PostgreSQL required for testing.

3. Frontend Setup

3.1 Install Dependencies

```
cd ../frontend
npm install
```

3.2 Create Environment File

```
cp .env.example .env
```

```
VITE_API_URL=http://localhost:5000/api/v1  
VITE_DEMO_MODE=false
```

3.3 Start Frontend

```
# Development  
npm run dev
```

Frontend will be available at: `http://localhost:5173`

3.4 Build for Production

```
npm run build
```

Output goes to `dist/` folder.

4. Production Deployment

4.1 PostgreSQL Production Setup

Use a managed PostgreSQL service: - **Railway** — one-click PostgreSQL addon
- **Supabase** — free tier available - **AWS RDS** — for enterprise (recommended for Saudi Arabia data residency) - **Neon** — serverless PostgreSQL

Get the connection string in format:

```
postgresql://user:password@host:5432/dbname?sslmode=require
```

4.2 Backend on Railway

1. Push code to GitHub
2. Go to Railway → New Project → Deploy from GitHub repo
3. Select the `backend/` directory as the service root
4. Add all environment variables in Railway dashboard
5. Set `NODE_ENV=production`
6. Railway auto-deploys on every push to main

4.3 Frontend on Vercel

1. Go to Vercel → New Project → Import GitHub repo
2. Set root directory to `frontend/`
3. Set build command: `npm run build`
4. Set output directory: `dist`
5. Add environment variable: `VITE_API_URL=https://your-railway-backend.railway.app/api/v1`
6. Set `VITE_DEMO_MODE=false`
7. Deploy

4.4 CORS Configuration

In the backend `.env`, set `CLIENT_URL` to your Vercel frontend URL:

```
CLIENT_URL=https://your-app.vercel.app
```

4.5 Verify Production

```
# Health check
curl https://your-backend.railway.app/api/v1/health

# Expected response
{"success": true, "data": {"status": "ok"}}
```

5. First-Time Setup After Deployment

Create First Organization

```
POST /api/v1/auth/register-org
{
  "organizationName": "Your Organization",
  "adminName": "Admin Name",
  "adminEmail": "admin@yourorg.com"
}
```

Create First User

Users self-register via the OTP flow:

```
POST /api/v1/auth/request-otp
{ "email": "user@example.com" }
```

```
POST /api/v1/auth/verify-otp
{ "email": "user@example.com", "otp": "123456" }
```

6. Environment Variables Reference

Variable	Required	Description
NODE_ENV	Yes	development / production / test
PORT	Yes	API server port (default 5000)
DB_URL	Yes	PostgreSQL connection string
JWT_SECRET	Yes	Min 64 chars random string
JWT_REFRESH_SECRET	Yes	Min 64 chars random string
JWT_EXPIRES_IN	Yes	Access token TTL (e.g. 15m)
JWT_REFRESH_EXPIRES_IN	Yes	Refresh token TTL (e.g. 7d)
OTP_SECRET	Yes	OTP signing secret
SMTP_HOST	Yes	SMTP server host
SMTP_PORT	Yes	SMTP port (587 for TLS)
SMTP_USER	Yes	SMTP username
SMTP_PASS	Yes	SMTP password / API key
EMAIL_FROM	Yes	Sender email address
CLIENT_URL	Yes	Frontend URL for CORS

7. Swagger API Documentation

Interactive API docs are available at:

```
http://localhost:5000/api/docs
```

The full OpenAPI specification is at:

```
docs/openapi.yaml
```

| Troubleshooting

Database connection fails - Verify PostgreSQL is running: `pg_isready` -
Check DB_URL format: `postgresql://user:pass@host:port/dbname` -
Ensure the database and user exist

OTP emails not received - Check SMTP credentials in `.env` - In development, OTP is logged to the console and returned in the response for testing - Test SMTP: `node -e "require('./src/utils/email').sendOtp('test@example.com', '123456')"`

CORS errors in browser - Ensure `CLIENT_URL` matches your exact frontend URL including protocol - No trailing slash in `CLIENT_URL`

Tests failing - Tests run against SQLite, not PostgreSQL — no DB setup needed for tests - Run: `NODE_ENV=test npm test`